Data Science Collective

Member-only story

Sparse MoE vs Speculative Decoding: The Fastest Way to Run a Local Coding LLM on Apple Silicon

Manjunath Janardhan Follow 10 min read · 1 day ago

I benchmarked three ways to run a local agentic coding assistant on a Mac. The mixture-of-experts model reached the first token up to 6× faster than the alternatives — and it has nothing to do with the trick everyone reaches for first.

Image by Manjunath Janardhan

Part 2 of a two-part series. In [Part 1 — Run Claude Code Locally on a Mac]

Run Claude Code Locally on a Mac: 65 tok/s with a 4-bit Qwen3.6–27B and…

A step-by-step guide to driving Claude Code entirely offline on Apple Silicon —…

medium.com

I wired up Claude Code to run entirely on a local 4-bit Qwen3.6–27B with DFlash speculative decoding. This is what happened when I tried to make it faster.

In Part 1, I got Claude Code running offline against a 4-bit Qwen3.6–27B on my Mac, using **DFlash speculative decoding** to push token generation to ~65 tok/s. It felt fast. So I went for the next speedup — and discovered I'd been optimizing the wrong half of the problem.

If you run a coding assistant like Claude Code, Aider, or

Cursor against a **local LLM on Apple Silicon**, you've felt the lag: you send a message, and the model just… sits there before a single token appears. The instinct is to reach for **speculative decoding** to fix it — exactly what Part 1 did.

So I tested that instinct properly. On an Apple Silicon Mac (M4, 64 GB unified memory), I ran the same agentic coding workload three ways and measured every turn. The result was the opposite of what I expected — and it points to a much simpler lever than spec-decode.

TL;DR — Local agentic coding is **prefill-bound**, not decode-bound. A **sparse mixture-of-experts (MoE)** model reaches first token up to **6.3× faster** than a dense model and **4.5× faster** than dense + speculative decoding at a realistic 24k-token context. Speculative decoding makes *token generation* 2–3× faster — but generation isn't the bottleneck, so it loses where it counts.

Why local agentic coding is prefill-bound

Two phases decide how fast an LLM feels:

- **Prefill** — the model reads your prompt and produces the *first* token. Cost scales with prompt length (time-to-first-token, TTFT).

- **Decode** — the model generates each *subsequent* token, one at a time.

For a chatbot, you type a short question and read a long answer, so **decode** dominates. Agentic coding is the mirror image. Tools like Claude Code stuff the system prompt, tool definitions, file contents, and conversation history into **every single turn** — easily 18,000–25,000 tokens — and the model often replies with a short tool call. So the cost is almost entirely **prefill**, paid again on every turn.

This matters because **speculative decoding only accelerates decode.** It drafts several tokens with a small model and verifies them in one pass with the big one. If your bottleneck is reading a 24k-token prompt, spec-decode is optimizing the wrong half.

The three contenders

All three are Qwen3.6-family models, quantized to run comfortably in 64 GB, served behind an OpenAI-compatible endpoint:

The MoE has *more* total parameters (35B vs 27B), but only activates ~3B per token. That single fact turns out to decide everything.

The setup

I deliberately **bypassed the agent loop** for measurement. Instead of driving Claude Code by hand — noisy and impossible to reproduce — I hit each server's `/v1/chat/completions` endpoint directly with a small harness that streams the response and records TTFT-based prefill tok/s, decode tok/s, and per-turn latency.

Two modes:

1. **Task mode** — the goal *"build a full-stack FastAPI + React calculator"* split into three small, focused sub-prompts (backend, component, wiring), each sent as a fresh single-turn request. This mimics good prompt hygiene: short, `/clear`'d turns.

2. **Sweep mode** — a realistic agentic system preamble padded to **2k / 8k / 16k / 24k tokens**, then a tiny question. This isolates the **prefill-vs-context curve**— the thing that actually hurts in agentic coding.

The two TurboQuant models were served with `turboquant-serve` (3-bit weights + 8-bit-key / 3-bit-value KV-cache quantization). The 4-bit model ran on `dflash serve` with prefix caching disabled, so every measurement is a clean full prefill.

GitHub - manjunathshiva/turboquant-mlx: Extreme weight + KV cache…

Extreme weight + KV cache compression for LLMs on Apple Silicon (MLX…

github.com

Result #1: Prefill vs context

This is the table that matters. **TTFT is how long you wait before the model starts responding** on each turn.

A sparse MoE prefills 4–6× faster than dense or speculative-decoding LLMs across 2K–24K context on an Apple Silicon Mac.

Prefill throughput by model and context length. The MoE (green) towers over speculative decoding (blue) and the dense model (grey) — and barely sags as context grows. Higher is better.

**prefill throughput · time-to-first-token*

At a realistic **24k-token agentic context**, the MoE reaches the first token in **31 seconds**, versus **141 seconds** for dense + speculative decoding and **195 seconds** for the plain dense model. That's the MoE being **~4.5× faster than spec-decode and ~6.3× faster than dense** — on the exact metric agentic coding lives and dies by.

Log-scale bar chart of time-to-first-token; the MoE reaches first token in 31s versus 141s and 195s at 24K-token context.

The same data as the wait you actually feel (log scale, lower is better). At 24K tokens the gap is the difference between a 31-second pause and a 2–3 minute one — every turn.

Notice the MoE's prefill barely sags as context grows (867 → 770 t/s). Sparse activation keeps per-token compute cheap, no matter how long the prompt gets.

Result #2: Decode — the plot twist

Flip to **decode** (token-generation speed) and the ranking inverts completely:

Bar chart showing speculative decoding has the fastest token generation but loses on prefill-bound agentic coding.

Decode throughput, higher is better. Speculative decoding (blue) wins this chart decisively — but decode isn't the agentic bottleneck, so winning it doesn't win the turn.

Speculative decoding is *spectacular* at decode — 145–182 tokens/sec, the fastest of the three by a wide margin. If you were running a chatbot, dflash would win going away.

But in agentic coding the replies are short, so this advantage barely moves the per-turn clock. **Spec-decode optimizes the half of the problem that isn't the bottleneck.** That's the whole story in one sentence.

Result #3: The `/clear` punchline

Here's the subtlety that explains why so many people misdiagnose this. On the tiny `/clear`'d sub-prompts (49–65 tokens), the per-turn ranking **inverts again**:

At tiny context, there's almost no prefill to pay, so turns are **decode-dominated** — and dflash's fast decode makes it feel snappiest.

This is the trap. If you test your local setup with short prompts, speculative decoding looks like the winner. The MoE's decisive advantage only appears once context grows and prefill takes over — which is exactly what happens the moment you point a real agent at a real codebase. Small prompts *hide* the gap; agentic prompts *expose* it.

Note: It's also a great argument for prompt hygiene: `/clear` often. A `/clear`'d MoE turn finishes in ~3s; the same model on a cold 24k context takes ~31s to first token.

Why the MoE wins: the mechanism

It comes down to one number: **active parameters per token.**

- The dense 27B activates **all 27B parameters** for every token of prefill.

- The MoE 35B-A3B routes each token to a handful of experts and activates only **~3B parameters**.

Prefill is a giant matrix-multiply over the prompt. Doing it with ~3B active parameters instead of 27B is roughly an order of magnitude less compute — which is exactly why the MoE prefills 6× faster despite being a *larger* model on disk. Speculative decoding can't compete here because it doesn't reduce prefill compute at all; it only saves verifier passes during generation.

Two nuances worth knowing

1. 4-bit prefills faster than 3-bit — at equal size. The dflash 4-bit model prefilled at 218 t/s vs the dense 3-bit model's 131 t/s at 2k (~1.7× faster). That's not the spec-decode draft — it's the quantization scheme. My **TurboQuant** 3-bit path applies an online Hadamard rotation per layer, which costs real compute during prefill. Standard 4-bit affine skips it. (The 3-bit win is memory footprint and quality-per-bit, not prefill speed.) Above 16k, dflash's prefill bends down (222 → 171 t/s) as $O(n^2)$ attention starts to bite.

2. KV-cache quantization is a memory win, not a speed win. I served the TurboQuant models with 8-bit-key / 3-bit-value KV-cache quantization, which keeps the cache tiny (well under 2 GB even at long context) so you don't OOM on a 64 GB Mac. It doesn't make prefill or decode faster — it makes long context *possible*.

From benchmark to a 4-minute stall:

While recording the Part 1 demo, I asked the local 4-bit 27B (the dflash setup) to *build and run* a full-stack FastAPI + React calculator app. It did. Then I asked it to stop the two dev servers — and it went silent for **four minutes.**

Part 1 Demo : four minutes lag.

It hadn't crashed. The conversation had grown to **35,129 tokens** — every file written, every tool result, plus the running servers' console output. That turn missed the prefix cache, so the dense 27B re-prefilled all 35k tokens at ~143 tok/s ≈ **245 seconds** before it could respond. (Memory was never the issue — it peaked at ~21 GB.)

This is the benchmark playing out in real life. Here's what that exact 35,129-token cold prefill costs on each setup:

The MoE doesn't make a cache miss *disappear*. It makes it **survivable** — a ~45-second annoyance instead of a four-minute stall. When you're driving an agent that occasionally blows past its prefix cache (and it will — long-lived dev-server output is the fastest way to balloon a context), that 5× margin is the difference between "usable" and "I gave up and hit `/clear`."

Two takeaways the stall makes concrete: **`/clear` before an unrelated request** (asking a 35k-token conversation to "stop the servers" re-reads all 35k first), and **a sparse MoE buys you headroom for the times you forget.**

How to reproduce this

Install the server and point it at a quantized MoE:

```
pip install -U turboquant-mlx-full

# Serve a sparse MoE with quantized weights AND a quantiz
turboquant-serve \
 -model manjunathshiva/Qwen3.6-35B-A3B-tq3-g32 \
 -kv-k-bits 8 -kv-v-bits 3 -kv-min-tokens 128 \
 -prompt-concurrency 1 \
```

```
  -port 8787 \
  -chat-template-args '{"enable_thinking": false}'
```

Then bridge it to your agent (e.g. via claude-code-router for Claude Code) — Part 1 walks through that wiring step by step — and **keep your context short**: `/clear` between unrelated tasks is the single biggest speedup available to you, for free.

Caveats

- **This measures speed, not quality.** A 3-bit MoE and a 4-bit dense model are different on coding accuracy; that's a separate study. Here I held the workload fixed and varied only the serving strategy.

- **Single-user, single-stream.** No batching. KV-quant forces sequential serving, which is correct for a personal coding assistant but not for a shared endpoint.

- **64 GB Apple Silicon, MLX backend.** Your absolute numbers will differ; the *ratios* should hold, because they're driven by active-parameter count, not the specific chip.

The takeaway

For **local agentic coding on Apple Silicon**, the speed lever isn't the one everyone grabs:

1. **Pick a sparse MoE.** Active-parameter count, not total size, decides prefill speed — and prefill is the bottleneck.

2. **Don’t count on speculative decoding.** It makes decode fly, but decode isn’t where the wait lives.

3. **Quantize the KV cache** to make long context fit, and **`/clear` aggressively** to keep prefill cheap.

A sparse MoE you can run on a Mac will *feel* faster than a dense model twice its apparent sophistication — not because it’s smarter, but because it reads your enormous agent prompt with a tenth of the compute.

Resources

- TurboQuant paper https://arxiv.org/abs/2504.19874 — Zandieh, Han, Daliri, Karbasi (2025).
- TurboQuant-MLX on PyPI https://pypi.org/project/turboquant-mlx-full — `pip install “turboquant-mlx-full>=0.4.1”`
- Qwen3.6–35B-A3B-tq3-g32 on HuggingFace https://huggingface.co/manjunathshiva/Qwen3.6-35B-A3B-tq3-g32 — the 35B streaming model card.
- TurboQuant-MLX on GitHub https://github.com/manjunathshiva/turboquant-mlx — Source, issues, Apache-2.0.
- DFlash project page (Z Lab) https://z-lab.ai/projects/dflash/
- DFlash reference implementation on GitHub: https://github.com/z-lab/dflash
- mlx-community/Qwen3.6–27B-4bit on Hugging Face https://huggingface.co/mlx-community/Qwen3.6-27B-4bit
- Draft — z-lab/Qwen3.6–27B-DFlash on Hugging Face https://huggingface.co/z-lab/Qwen3.6-27B-DFlash

Support

If you found this article informative and valuable, I'd greatly appreciate your support:

"Give it a few claps 👏 on Medium to help others discover this content (did you know you can clap up to 50 times?). Your claps will help spread the knowledge to more readers."

- Share it with your network of AI enthusiasts and professionals.
- Subscribe to my YouTube channel for AI videos explained in simple English: https://www.youtube.com/@AIBroEnglish
- Connect with me on LinkedIn: https://www.linkedin.com/in/manjunath-janardhan-54a5537/

Thanks For Reading

Speculative Decoding | Moe | Apple Silicon | Claude Code

Published in Data Science Collective

Follow

931K followers · Last published 1 day ago

Advice, insights, and ideas from the Medium data science community

Written by Manjunath Janardhan

Follow

1.1K followers · 87 following

AI/ML Computational Science Senior Manager at Accenture with 21+ years building enterprise AI, GenAI, and intelligent operations.